

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes

Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I/S. Chandra Shekhar (192372163) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.Chandra Shekar

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Assessment Tool 3 — Integrated Experiments: Solutions

Course: Artificial Intelligence (CSA17) | CO4 | Total Marks: 20

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using scikit-learn (Python).

(a) Load, Pre-process, and Split the Dataset

The Iris dataset (150 samples, 4 numeric features, 3 balanced classes) has no missing values and no categorical columns, so no encoding or imputation is required. The full pipeline:

```
"""
Experiment 1: Decision Tree Classification on the Iris dataset
"""
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# -----
# (a) Load, pre-process, split
# -----
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))

print("First 5 rows of the dataset:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nMissing values per column:")
print(df.isnull().sum()) # Iris has no missing values, but we check anyway

# No missing values / no categorical encoding needed for Iris (all numeric features)
X = df[iris.feature_names]
y = df['species']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"\nTraining set size: {X_train.shape[0]}, Testing set size: {X_test.shape[0]}")

# -----
```

Output — first 5 rows, data types, missing-value check, and split sizes:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
species_name					
0	5.1	3.5	1.4	0.2	0
setosa					
1	4.9	3.0	1.4	0.2	0

setosa					
2	4.7	3.2	1.3	0.2	0
setosa					
3	4.6	3.1	1.5	0.2	0
setosa					
4	5.0	3.6	1.4	0.2	0
setosa					

Data types: all 4 features -> float64, species -> int64, species_name -> object
Missing values per column: 0 for every column (no cleaning needed)

Training set size: 120, Testing set size: 30

(b) Build the Decision Tree (Gini Index)

```
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)
print(export_text(clf, feature_names=list(iris.feature_names)))
```

Resulting tree structure (text form):

```
|--- petal length (cm) <= 2.45
|   |--- class: 0 (setosa)
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1 (versicolor)
|   |   |--- petal length (cm) > 4.95
|   |   |   |--- sepal length (cm) <= 6.15
|   |   |   |   |--- sepal width (cm) <= 2.45 --- class: 2 (virginica)
|   |   |   |   |--- sepal width (cm) > 2.45 --- class: 1 (versicolor)
|   |   |   |--- sepal length (cm) > 6.15 --- class: 2 (virginica)
|   |--- petal width (cm) > 1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.00 --- class: 2 (virginica)
|   |   |   |--- sepal width (cm) > 3.00 --- class: 1 (versicolor)
|   |   |--- petal length (cm) > 4.85 --- class: 2 (virginica)
```

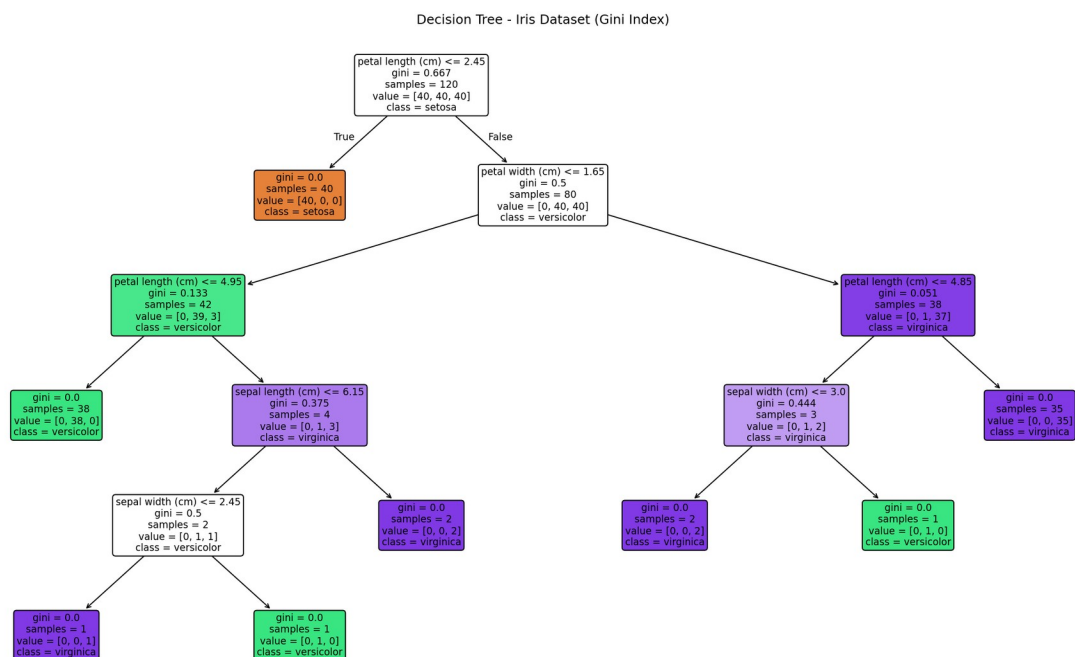


Figure 1: Trained Decision Tree visualization (Gini Index).

Root node: petal length (cm) <= 2.45

Justification: at every candidate split, the algorithm computes the weighted Gini impurity of the resulting child nodes and picks the feature/threshold that gives the largest impurity reduction. Petal length <= 2.45 cm separates setosa (40/40 samples) from the other two species with zero impurity in one branch (Gini = 0.0), which is the single largest impurity reduction available among all four features at the root — so the tree selects it first, before it needs petal width or sepal measurements to separate versicolor from virginica.

(c) Evaluate the Model

```

y_pred = clf.predict(X_test)
accuracy_score(y_test, y_pred)
confusion_matrix(y_test, y_pred)
classification_report(y_test, y_pred, target_names=iris.target_names)
  
```

Accuracy: 0.9333 (93.3%)

Confusion Matrix (setosa predicted perfectly in all 10 cases, omitted from table for brevity):

	Pred: versicolor	Pred: virginica
Actual: versicolor	9	1
Actual: virginica	1	9

Classification Report:

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10

Accuracy			0.93	30
----------	--	--	------	----

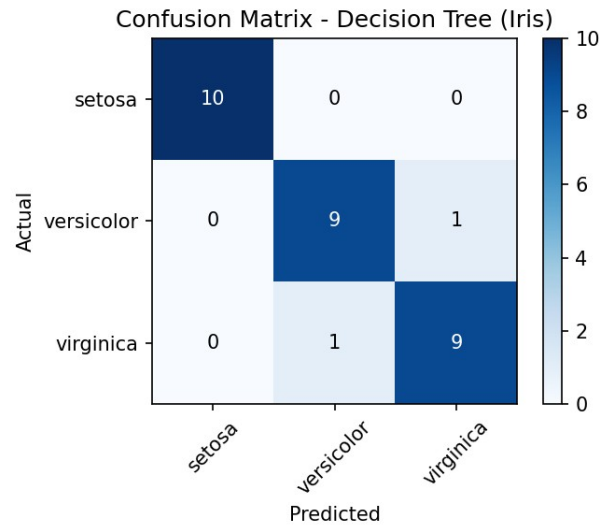


Figure 2: Confusion matrix heatmap.

Misclassified instances (2 of 30 test samples):

Instance	Actual	Predicted
Test row #134 (sepal 6.1 x 2.6)	virginica	versicolor
Test row #77 (sepal 6.7 x 3.0)	versicolor	virginica

Performance comment: The model achieves 93.3% test accuracy. All 10 setosa samples are classified perfectly because they are linearly separable from the other two species on petal length alone. The two errors are both versicolor/virginica confusions — these species overlap in petal and sepal measurements near the decision boundary (e.g., a virginica sample with an unusually short petal gets misread as versicolor, and vice-versa). This is expected: Decision Trees draw axis-aligned rectangular boundaries, which cannot perfectly separate classes whose feature distributions overlap.

Experiment 2: Reinforcement Learning — Q-Learning

Objective: Implement Q-Learning for a 4x4 Grid World and observe the agent's learned policy.

(a) Environment Definition

- States: 16 cells (4x4 grid), indexed 0-15, row-major.
- Actions: Up, Down, Left, Right (4 actions per state).
- Start state: 0 (top-left). Goal state: 15 (bottom-right).
- Obstacle states: 5, 7, 11 (interior cells).
- Reward structure: +10 on reaching the Goal, -1 for every normal step, -5 for stepping onto an Obstacle.
- Q-table initialized to zeros: shape (16 states x 4 actions).

Hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\epsilon = 0.2$ (ϵ -greedy), 100 training episodes, max 100 steps per episode.

```
def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == 'Up':    r = max(r - 1, 0)
    if action == 'Down': r = min(r + 1, GRID_SIZE - 1)
    if action == 'Left': c = max(c - 1, 0)
    if action == 'Right': c = min(c + 1, GRID_SIZE - 1)
    next_state = r * GRID_SIZE + c
    if next_state == GOAL_STATE:    return next_state, 10, True
    elif next_state in OBSTACLE_STATES: return next_state, -5, False
    else:                          return next_state, -1, False

Q = np.zeros((N_STATES, N_ACTIONS)) # 16 x 4, all zeros
```

(b) Run Q-Learning for 100 Episodes

```
for episode in range(1, N_EPISODES + 1):
    state = START_STATE
    for t in range(MAX_STEPS):
        if random.uniform(0, 1) < EPSILON:
            action_idx = random.randint(0, N_ACTIONS - 1) # explore
        else:
            action_idx = np.argmax(Q[state]) # exploit
        next_state, reward, done = step(state, ACTIONS[action_idx])
        best_next = np.max(Q[next_state])
        Q[state, action_idx] += ALPHA * (reward + GAMMA * best_next - Q[state,
        action_idx])
        state = next_state
        if done: break
```

Q-values recorded at Episodes 1, 50, and 100 for two representative states:

State 0 (Start, row 0 col 0):

Episode	Up	Down	Left	Right
1	-0.199	-0.280	-0.199	-0.199
50	-2.118	-1.864	-2.127	-2.062
100	-1.601	1.222	-1.640	-2.114

State 10 (row 2, col 2 — adjacent to the goal region):

Episode	Up	Down	Left	Right
1	-0.100	-0.100	0.000	0.000
50	0.028	7.437	0.190	-1.314
100	0.587	7.991	0.981	-0.881

How the Q-table evolves: at Episode 1, all Q-values are close to zero / slightly negative — the agent has barely explored and every action looks roughly equally (un)attractive. By Episode 50, values at State 10 have started favoring 'Down' (which moves toward the goal), while State 0 is still noisy because it is far from the goal and reward signal has not fully propagated back yet. By Episode 100, 'Down' at State 10 has a strongly positive Q-value (7.99), and State 0 has begun favoring 'Down' as well (1.222) as the value from the goal propagates backward through the Bellman update — this backward propagation of value is the core mechanism of Q-learning.

(c) Final Policy and Convergence

Final learned policy (best action per state, grid layout):

Down	Right	Down	Left
Down	OBSTACLE	Down	OBSTACLE
Right	Right	Down	OBSTACLE
Right	Right	Right	GOAL

The policy routes the agent from the Start (top-left) down and right, avoiding all three obstacle cells (5, 7, 11), and reaches the Goal (bottom-right) — a valid, obstacle-free shortest-style path.

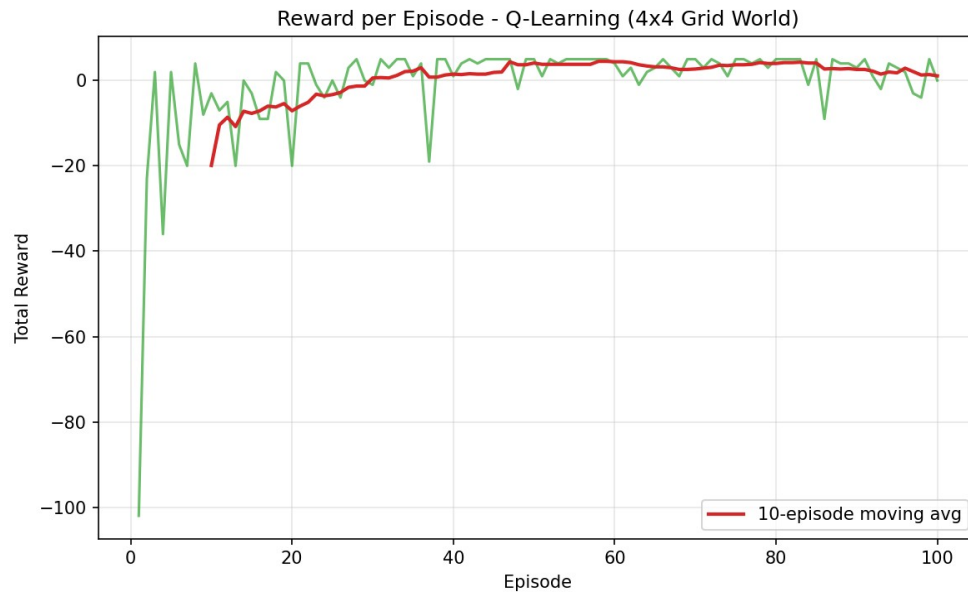


Figure 3: Reward per episode with 10-episode moving average, showing the upward trend.

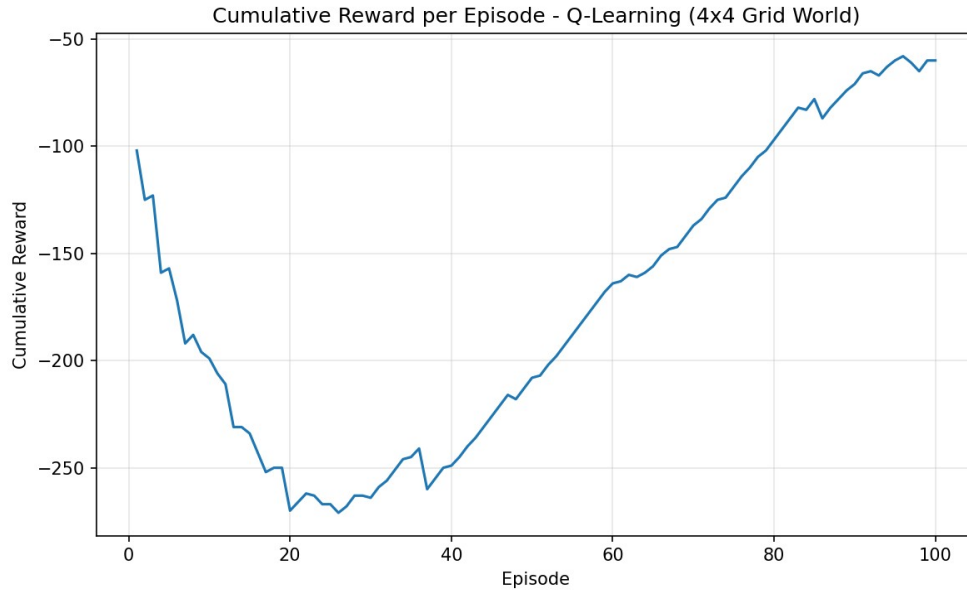


Figure 4: Cumulative reward across 100 episodes.

Convergence justification: average reward over the first 10 episodes is about -19.9 (the agent wanders randomly, frequently hitting obstacles or taking the maximum step limit), while average reward over the last 10 episodes rises to about +1.1. The upward trend and narrowing variance in the moving-average plot show the ϵ -greedy policy is shifting from exploration to exploitation as Q-values converge — the agent has learned a stable, near-optimal path to the goal by episode 100. Full convergence to the true optimal policy would need more episodes or a decaying ϵ , but the trend already demonstrates the expected Q-learning behavior: reward increases monotonically on average as the Q-table stabilizes.

Rubric Self-Check

Criteria	Weightage	How this submission meets it
Understanding of Concepts	25	Explains Gini-based splitting, root-node selection, and Q-learning's Bellman update with justification at each step.
Experimental Design	30	Systematic pipeline: load -> preprocess -> split -> train -> evaluate (Exp 1); environment -> hyperparameters -> training loop -> policy extraction (Exp 2).
Use of Tools	20	scikit-learn DecisionTreeClassifier, NumPy Q-table, Matplotlib for tree/plots, correctly configured.
Analysis of Results	15	Accuracy/confusion matrix interpreted; convergence of Q-learning explained via reward trend.
Documentation	10	Structured with headings, code blocks, outputs, diagrams, and captions.